

Technical proposal: hybrid Databricks integration for CHEC Local UITI_VANO Interpreter

Decision

The most appropriate architecture is a **hybrid model**:

- **Databricks** becomes the system of record for data, feature pipelines, re-training, model registry, model serving, and operational dashboards.
- **This repository running in Pi / el Gentleman** remains the agent-native orchestration layer for `/skill:report`, role execution, validation, and final HTML report generation.

This proposal deliberately avoids a full migration of Pi skills, subagents, Gentle AI harnesses, and Engram-style memory into Databricks during the first implementation stage.

1. Objectives

Business objectives

1. Centralize data and model operations in Databricks.
2. Enable controlled retraining and governed model promotion.
3. Serve model outputs and explainability artifacts from a managed platform.
4. Preserve the current report-generation workflow used by analysts/operators.
5. Avoid a disruptive rewrite of the agent runtime before proving value.

Technical objectives

1. Decouple this repo from local-only data/model assets.
2. Introduce stable integration contracts between Pi runtime and Databricks services.
3. Add end-to-end traceability between reports, model versions, and data snapshots.
4. Preserve existing deterministic validation boundaries.
5. Keep the migration reversible during the first phases.

Non-goals

1. Replacing Pi/el Gentleman with Databricks-native agents in phase 1.
2. Rewriting role prompts, skills, or validators around Mosaic AI immediately.
3. Moving bounded review, subagent orchestration, or harness semantics into Databricks.
4. Publishing reports automatically as part of the first integration slice.

2. Why the hybrid architecture is the right fit

The repo architecture is already strongly layered:

- deterministic Python prepares context
- agent-facing tool CLIs expose controlled envelopes
- the coding-agent runtime performs reasoning
- validators enforce output structure and provenance
- `/skill:report` orchestrates the end-to-end workflow

This means the project is not just a model consumer. It is a **controlled analytical runtime**.

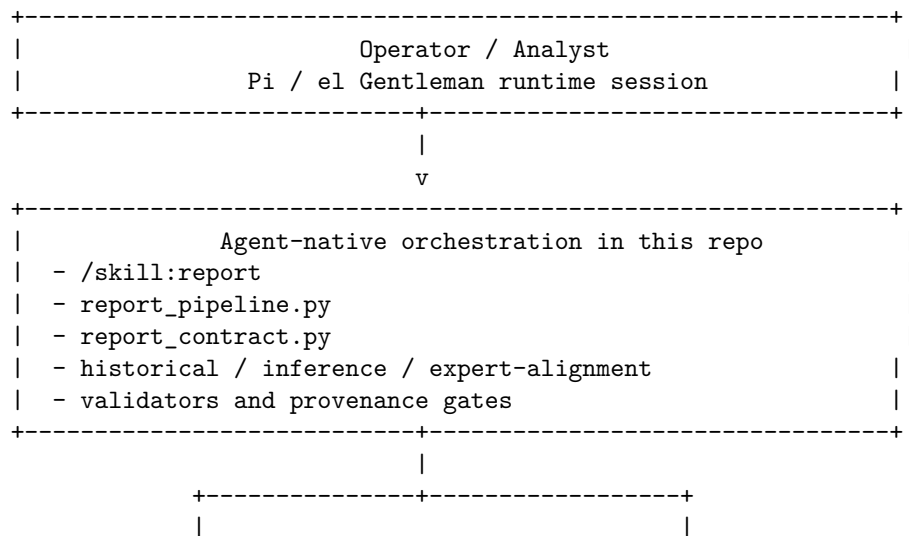
Databricks is excellent for: - Delta / Unity Catalog - scheduled jobs - MLflow and model registry - model serving - feature management - operational dashboards

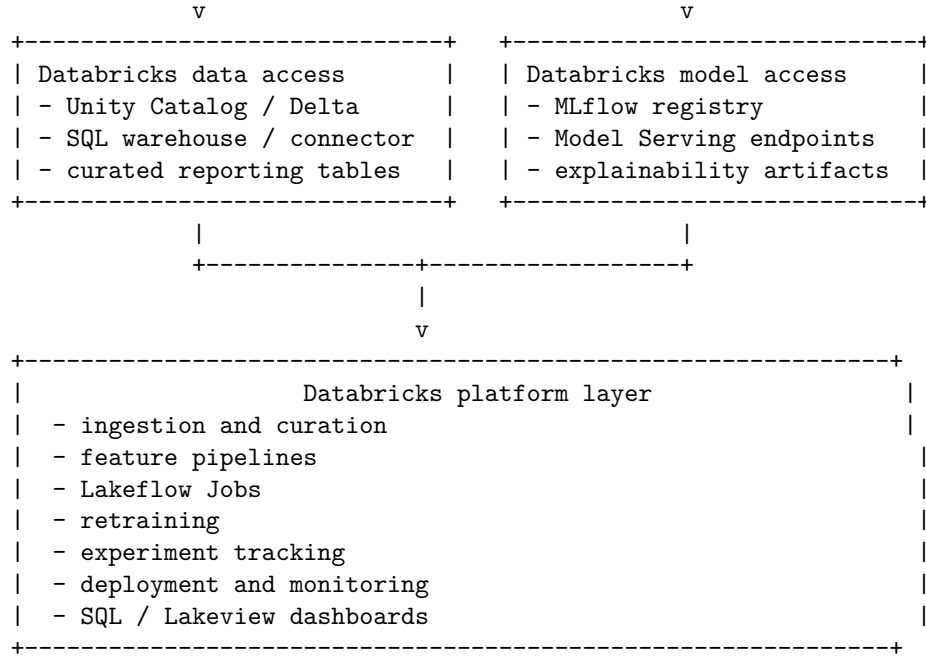
But Databricks is not a natural drop-in replacement for: - Pi markdown skills
- subagent execution - Gentle AI review/harness semantics - Engram memory
contracts - interactive command endpoints such as `/skill:report`

So the correct move is to migrate the **backend platform responsibilities**, not the **operator-facing agent runtime**.

3. Target architecture

3.1 Logical architecture





3.2 Runtime responsibility split

Concern	Databricks	This repo / Pi
Raw and curated data	Yes	No
Training and retraining	Yes	No
Model registry	Yes	Read-only consumption
Online inference endpoint	Yes	Calls endpoint
Explainability batch artifacts	Yes	Consumes
Context assembly for roles	Optional source	Yes
LLM/agent reasoning	No	Yes
Validation/provenance gates	No	Yes
Final HTML report rendering	No	Yes
Operator UX (/skill:report)	No	Yes

4. Integration design

4.1 New integration components in this repo

Recommended new modules:

```
src/chec_local_interpreter/
  databricks/
```

```
__init__.py
config.py
auth.py
data_source.py
sql_client.py
model_registry.py
serving_client.py
artifact_loader.py
lineage.py
cache.py
exceptions.py
```

Module responsibilities

`config.py`

- Resolve Databricks host, warehouse id, catalog, schema, serving endpoint names, and feature flags.
- Read from environment variables or a local config file excluded from git.

`auth.py`

- Centralize authentication strategy.
- Prefer service principal / PAT / OAuth depending on enterprise policy.
- Never spread token handling across pipeline modules.

`sql_client.py`

- Low-level SQL/connector wrapper.
- Execute parameterized queries.
- Return pandas DataFrames or typed records.

`data_source.py`

- High-level domain queries for circuit/date windows.
- Provide methods such as:
 - `load_circuit_slice(circuito, fecha_inicio, fecha_fin)`
 - `load_variable_metadata()`
 - `load_relationship_rules()`
 - `load_pdf_alignment_reference()`
- Normalize Databricks results into the shapes already expected by the current deterministic pipeline.

`model_registry.py`

- Read active model name, version, stage/alias, training metadata, and tags from MLflow.
- Persist model identity into run artifacts.

`serving_client.py`

- Call Databricks Model Serving endpoints.
- Handle request/response normalization, retries, and error mapping.
- Optional support for prediction plus explainability endpoints.

`artifact_loader.py`

- Load SHAP summaries, scenario outputs, or derived artifacts from Databricks-managed storage/tables.
- Keep artifact retrieval logic isolated from business orchestration.

`lineage.py`

- Build a per-run lineage record containing:
 - report run id
 - circuit
 - date window
 - table versions or snapshot timestamp
 - model name/version/alias
 - serving endpoint revision
 - generated artifact paths

`cache.py`

- Optional short-lived local cache for immutable reads during one run.
- Avoid repeating remote queries across multiple role stages.

4.2 Integration points in the current workflow

`report_pipeline.py`

Recommended changes:

1. **Preflight phase**
 - Validate Databricks connectivity when cloud mode is enabled.
 - Resolve available date ranges from Databricks instead of local files.
2. **Prepare phase**
 - Use `data_source.py` to fetch the reporting slice.
 - Use `model_registry.py` / `artifact_loader.py` to attach model metadata and explainability assets.
 - Persist lineage metadata into `run_dir`.
3. **Role context generation**
 - Keep existing context contracts stable.
 - Databricks integration must adapt data to the current expected schema instead of forcing agent contracts to change.
4. **Render phase**

- Add a traceability section to the generated report:
 - data snapshot
 - model version
 - endpoint name
 - generation timestamp

report_contract.py

Recommended changes: - Add optional metadata fields for Databricks-backed runs. - Keep backward compatibility so local mode still works.

Suggested metadata additions:

```
{
  "runtime": "pi",
  "backend": "databricks",
  "databricks": {
    "catalog": "...",
    "schema": "...",
    "source_table": "...",
    "snapshot_at": "...",
    "model_name": "...",
    "model_version": "...",
    "model_alias": "...",
    "serving_endpoint": "..."
  }
}
```

agent_tools/*

Minimal changes preferred: - do not make agent role CLIs aware of Databricks directly unless necessary - keep Databricks-specific concerns above them, in orchestration/preparation layers

This preserves the current architecture discipline.

5. Databricks-side design

5.1 Data platform layout

Recommended medallion-style organization:

Bronze

- raw circuit and operational data
- minimally transformed ingestion

Silver

- cleaned and standardized time-series tables
- normalized circuit identifiers
- typed columns and validated timestamps

Gold

- report-ready datasets by circuit/date
- feature tables for retraining
- explainability summary tables
- report support lookup tables

Recommended gold datasets: - `gold.circuit_daily_series` - `gold.circuit_variable_metadata`
- `gold.circuit_relationship_rules` - `gold.report_context_ready` -
`gold.inference_explainability_summary`

5.2 Training and retraining

Recommended flow:

curated data -> feature engineering -> training job -> evaluation -> MLflow register -> app

Use Databricks for: - scheduled retraining - experiment tracking - model comparison - stage/alias assignment - deployment automation guarded by approval

5.3 Serving

Recommended endpoint split:

Endpoint A — prediction/inference

- primary model outputs
- scenario scoring if applicable

Endpoint B — explainability

- SHAP summaries or explanation payloads
- may be precomputed batch output instead of live serving if latency/cost is high

The report runtime should not need to know how these are computed internally. It should consume a stable contract.

5.4 Dashboards

Use Databricks SQL or Lakeview for: - retraining status - latest promoted model - data freshness - endpoint health - report usage summaries if exported back to Databricks

6. Contracts and schemas

6.1 Principle

Do not redesign role prompts first.

The integration layer must map Databricks outputs into the existing internal contracts. That keeps the migration small and testable.

6.2 Adapter contract

Example normalized structure delivered by `data_source.py`:

```
{
  "circuito": "DON23L13",
  "periodo": {
    "inicio": "2026-01-01",
    "fin": "2026-01-31"
  },
  "series": [...],
  "metadata": {...},
  "variables": {...},
  "domain_rules": {...},
  "source": {
    "backend": "databricks",
    "table": "gold.report_context_ready",
    "snapshot_at": "2026-02-01T10:00:00Z"
  }
}
```

6.3 Traceability contract

Every run should persist: - report run id - backend mode - source table(s) - query timestamp or snapshot/version - model name/version/alias - serving endpoint - generated report path

This should be saved both: - inside the run directory - in the rendered report metadata

7. Security and access model

7.1 Authentication

Recommended order of preference:

1. service principal with scoped permissions
2. OAuth machine-to-machine where enterprise-supported
3. PAT only for early development or emergency fallback

7.2 Authorization

Principles: - read-only access from Pi runtime to reporting datasets - read-only access to MLflow registry metadata - invoke-only access to serving endpoints - no broad workspace admin rights for the reporting runtime

7.3 Secrets

- Keep secrets outside the repo.
 - Use environment variables or enterprise secret management.
 - Do not embed tokens in notebooks, markdown, or run artifacts.
-

8. Reliability design

8.1 Failure domains

Separate these failure classes clearly:

1. Databricks connectivity/auth failure
2. missing or stale data snapshot
3. model registry resolution failure
4. serving endpoint timeout/failure
5. local orchestration or validation failure
6. final render/export failure

8.2 Error handling strategy

- Fail fast during preflight when connectivity or required metadata is unavailable.
- Cache resolved inputs per run to avoid inconsistent multi-stage reads.
- Convert remote failures into explicit typed exceptions.
- Write machine-readable run diagnostics in `run_dir`.

8.3 Resilience patterns

- retry idempotent reads
 - avoid retry storms on serving failures
 - prefer batch-precomputed explainability over expensive live explanation when possible
 - capture exact source metadata once and reuse it throughout the run
-

9. Implementation phases

Phase 1 — Foundation

Scope

- add Databricks config/auth/adapters
- add backend mode flag
- keep local mode working unchanged

Deliverables

- `databricks/` module skeleton
- configuration contract
- connectivity check command or preflight hook
- first domain read from Databricks for circuit/date slice

Acceptance criteria

- `/skill:report` can run in local mode and Databricks mode
- no role contracts changed yet
- run artifacts record backend mode

Phase 2 — Data-backed reporting

Scope

- use Databricks as source for report input data
- normalize into current context contracts

Deliverables

- data queries for circuit/date windows
- metadata and rules loaders
- schema normalization tests

Acceptance criteria

- generated contexts are equivalent enough for existing validators and role flow
- report output remains structurally unchanged

Phase 3 — Model and explainability integration

Scope

- fetch active model metadata from MLflow
- consume serving endpoint and/or precomputed explainability artifacts

Deliverables

- MLflow registry adapter
- serving client
- lineage metadata persisted in run artifacts

Acceptance criteria

- each report identifies the exact model version used
- failures in serving do not silently degrade into fake outputs

Phase 4 — Operationalization

Scope

- scheduled retraining and promotion workflows in Databricks
- optional export of report usage or report metadata back to Databricks

Deliverables

- jobs for retraining
- model promotion policy
- monitoring dashboards

Acceptance criteria

- retraining, promotion, and consumption are traceable end-to-end

Phase 5 — Strategic review

Scope

- decide whether a deeper Databricks-native agent rewrite is justified

Exit question

- Is there enough value in replacing Pi runtime behavior, or is the hybrid model already sufficient?

10. Testing strategy

Unit tests

- config parsing
- query normalization
- MLflow metadata mapping
- serving response normalization
- lineage record generation

Contract tests

- Databricks result -> current context package shape
- local mode vs Databricks mode equivalence on selected fixtures

Integration tests

- authenticated connectivity to a non-production workspace
- end-to-end report preflight with mocked or sandbox endpoints

Regression tests

- current validators still pass with Databricks-backed context
- rendered HTML still includes expected sections

Operational checks

- stale data detection
 - missing model alias handling
 - endpoint timeout handling
-

11. Observability

Each run should emit structured metadata such as:

```
{  
  "run_id": "...",  
  "backend": "databricks",  
  "circuito": "DON23L13",  
  "periodo": {"inicio": "...", "fin": "..."},  
  "data_snapshot": "...",  
  "model_name": "...",  
  "model_version": "...",  
  "serving_endpoint": "...",  
  "status": "ok"  
}
```

Recommended sinks: - local `run_dir` - optional Databricks audit table in later phases

12. Alternatives considered

Alternative A — Full migration to Databricks now

Rejected for first implementation.

Reason: - too much platform rewrite risk - weak fit for current Pi/Gentle AI semantics - slows down time to value

Alternative B — Databricks Apps as immediate front-end replacement

Rejected for first step.

Reason: - the current operator UX is command-oriented and agent-driven, not just a web form - Apps may later host dashboards or light operational UIs, but they should not replace the runtime before backend integration is proven

Alternative C — Move only training, keep all inference local forever

Rejected as final target.

Reason: - loses the value of managed serving, registry, and traceability - creates split-brain model operations long term

13. Final recommendation

Implement a **hybrid Databricks backend + Pi agent-runtime architecture**.

Recommended sequence

1. Keep `/skill:report` and current agent orchestration intact.
2. Introduce Databricks adapters behind stable interfaces.
3. Move data, retraining, registry, and serving to Databricks.
4. Add lineage and traceability to every report run.
5. Reevaluate full platform migration only after the hybrid model is stable.

This is the most appropriate path because it improves governance, scalability, and ML operations **without destroying the architectural strengths already present in this repository**.

Appendix A — Suggested environment variables

```
DATABRICKS_HOST=  
DATABRICKS_TOKEN=  
DATABRICKS_WAREHOUSE_ID=  
DATABRICKS_CATALOG=  
DATABRICKS_SCHEMA=  
DATABRICKS_REPORT_SOURCE_TABLE=
```

```
DATABRICKS_VARIABLE_METADATA_TABLE=  
DATABRICKS_RULES_TABLE=  
DATABRICKS_MLFLOW_MODEL_NAME=  
DATABRICKS_SERVING_ENDPOINT=  
CHEC_BACKEND_MODE=local|databricks
```

Appendix B — Suggested first deliverable slice

The safest first implementation slice is:

1. add backend mode configuration
2. implement Databricks connectivity preflight
3. query a circuit/date dataset from Databricks
4. normalize it into current pipeline input shape
5. generate one report without changing role contracts

That slice proves the architecture with low blast radius.